



SIMATS Online Programming Lab

JAVA PROGRAMMING



NITHISH RAAJU V
192411016RD

QUESTIONS

Q1

Multiple Threads Using a Single Class

10 marks



Q2

Multiple thread classes

20 marks



Q3

Thread Synchronization

10 marks



3/3 answered

Q2 Multiple thread classes

20 marks

Question:

Write a Java program to create three different thread classes to perform different operations on the same number.

Create the following classes:

PrimeThread → Check whether the given number is prime.

FactorialThread → Find the factorial of the given number.

ReverseThread → Find the reverse of the given number.

Each class should extend the Thread class and override the run() method.

Create and start all three threads from the main() method.

Input: 7

Expected Output:

Prime: 7 is a prime number

Factorial: 7! = 5040

Reverse: Reverse of 7 = 7

Note: Since the threads execute concurrently, the order of output may vary.

Your Code

```
1 import java.util.Scanner;
2
3 public class Main {
4     public static void main(String[] args)
5     {
6         Scanner sc = new Scanner(System.in);
7         if (!sc.hasNextInt()) {
8             sc.close();
9             return;
10        }
11        int n = sc.nextInt();
12        sc.close();
13
14        PrimeThread pt = new PrimeThread(n);
15        FactorialThread ft = new FactorialThread(n);
16        ReverseThread rt = new ReverseThread(n);
17
18        pt.start();
19        ft.start();
20        rt.start();
21
22        try {
23            pt.join();
24            System.out.println(pt.result);
25
26            ft.join();
27            System.out.println(ft.result);
28
29            rt.join();
30            System.out.println(rt.result);
31        } catch (InterruptedException e) {
32            System.out.println("Threads interrupted");
33        }
34    }
35
36    class PrimeThread extends Thread {
37        private final int n;
38        public String result = "";
39
40        PrimeThread(int n) {
41            this.n = n;
42        }
43
44        private boolean isPrime(int num) {
45            if (num < 2) return false;
46            for (int i = 2; i * i <= num; i++)
```

```
47         if (num % i == 0) return false;
48     }
49     return true;
50 }
51
52 @Override
53 public void run() {
54     if (isPrime(n)) {
55         result = "Prime: " + n + " is
56     } else {
57         result = "Prime: " + n + " is
58     }
59 }
60 }
61
62 class FactorialThread extends Thread {
63     private final int n;
64     public String result = "";
65
66     FactorialThread(int n) {
67         this.n = n;
68     }
69
70     private long factorial(int num) {
71         long fact = 1;
72         for (int i = 2; i <= num; i++) {
73             fact *= i;
74         }
75         return fact;
76     }
77
78 @Override
79 public void run() {
80     long fact = factorial(n);
81     result = "Factorial: " + n + "! =
82 }
83 }
84
85 class ReverseThread extends Thread {
86     private final int n;
87     public String result = "";
88
89     ReverseThread(int n) {
90         this.n = n;
91     }
92
93     private int reverse(int num) {
94         int rev = 0;
```

```
95         int temp = num;
96         while (temp != 0) {
97             rev = rev * 10 + (temp % 10);
98             temp /= 10;
99         }
100        return rev;
101    }
102
103    @Override
104    public void run() {
105        int rev = reverse(n);
106        result = "Reverse: Reverse of " +
107    }
108 }
```

Input

7

Output

Prime: 7 is a prime number
Factorial: 7! = 5040
Reverse: Reverse of 7 = 7

Visible Test Cases

- | | |
|-------------|---|
| Test Case 1 | ✓ |
| Test Case 2 | ✓ |
| Test Case 3 | ✓ |

20.00 / 20 marks

✓ Submitted